PyMuPDF & OCR Engine Benchmark Suite

Document Processing Test Sample • Document Layout Analysis & Extraction Standard

Document Objective

This benchmark document is engineered to test programmatic PDF processing toolkits (such as **PyMuPDF / FitZ**, **pdfplumber**, and **pypdf**) alongside computer vision OCR pipelines (such as **Tesseract**, **PaddleOCR**, or **AWS Textract**). It validates font encoding, multi-column boundary detection, and tabular extraction.

1. Optical Character & Glyph Parsing Verification

Font Variant & Style Sampling

Standard Serif: The quick brown fox jumps over the lazy dog.

Numbers & Symbols: 0123456789 • $£€¥ %#@&*

Ligatures & Diacritics: æsthetic, fi/fl processing, naïve, résumé.

Boundary & Metadata Checks

Punctuation Test: Em-dash (—), En-dash (–), Quotes (“” ‘’).

Mixed Density: UPPERCASE, lowercase, *Italics*, **Bold**.

Status Marker: ACTIVE TEST PASSED

2. Tabular Data Extraction Matrix

ID	Parser Module	Target Structure	Precision Target	Execution Status
MOD-01	PyMuPDF Text Blocks	Direct Vector Extraction	99.8%	Validated
MOD-02	OCR Optical Pipeline	Rasterized Page (300 DPI)	96.5%	In Progress
MOD-03	Table Border Finder	Grid & Cell Alignment	100.0%	Passed
MOD-04	Bounding Box Detector	BBox Coordinates [x0, y0, x1, y1]	98.2%	Validated

3. Complex Document Parsing & Layout Challenges

Document layout analysis algorithms often struggle with multi-column structures, variable margin callouts, and inline programmatic code spans. When running PyMuPDF or Tesseract, it is essential to verify that reading order flows logically from left to right and top to bottom, avoiding text collisions between distinct visual blocks.

4. Special Extraction Edge Cases

Subscripts & Superscripts: Chemical formulas like H_2O, CO_2, and mathematical powers $x^2 + y^2 = z^2$.

Edge Coordinate Precision: Top-left (18mm, 18mm), Bottom-right (192mm, 285mm).

Special Ambiguity Pairs: O vs 0 (Zero), l (Lowercase L) vs 1 (One) vs I (Capital i), S vs 5.

5. PyMuPDF Python Test Implementation

```
      import fitz # PyMuPDF library doc = fitz.open("pymupdf_ocr_test_en.pdf") print(f"Total Pages: {len(doc)}") for index, page in enumerate(doc): # Extract structural dictionary with bounding boxes text_page = page.get_text("blocks") print(f"--- Page {index + 1} Blocks Count: {len(text_page)} ---") for b in text_page[:2]: # Print first 2 blocks print(f"BBox: {b[:4]} | Content: {b[4][:40]}...")
```

6. Engineering & Evaluation Notes

If extracting text via PyMuPDF returns clean ASCII strings without garbled character codes, the underlying PDF contains properly formatted font encoding maps (ToUnicode). If extraction returns empty or corrupt strings, your pipeline must automatically fallback to a high-resolution OCR renderer (e.g. converting pages to 300 DPI PNG images before OCR processing).